# CS1312
# Algorithms Design and Analysis

## Greedy Graph Algorithms

## Part 2

# Shortest Path Problem

- In a shortest path problem, we are given:
  1. A weighted, directed graph G = (V, E)
  2. Weight function w: E → R mapping edges to real valued weights.
- The weight w(p) of path p is the sum of the weights of its constituent edges.
- A shortest path from vertex u to vertex v is defined as any path p with minimum weight.

# Single-source shortest path problem

Given a graph G (V, E), we want to find a shortest path from a given source vertex $s \in V$ to each vertex $v \in V$ .

# Single-source shortest path problem

- Some instances of the **single-source** shortest-paths problem may include edges whose weights are negative.
  - If the graph G (V,E) contains no negative-weight cycles reachable from the source s, then for all $v \in V$ , the shortest-path weight remains well defined, even if it has a negative value.
  - If the graph contains a negative-weight cycle reachable from s, however, shortest-path weights are not well defined. **No** path from **s** to a vertex on the cycle can be the shortest

# Shortest Path Problem (Cont.)

- Negative cycle: directed cycle whose sum of edges costs is negative.

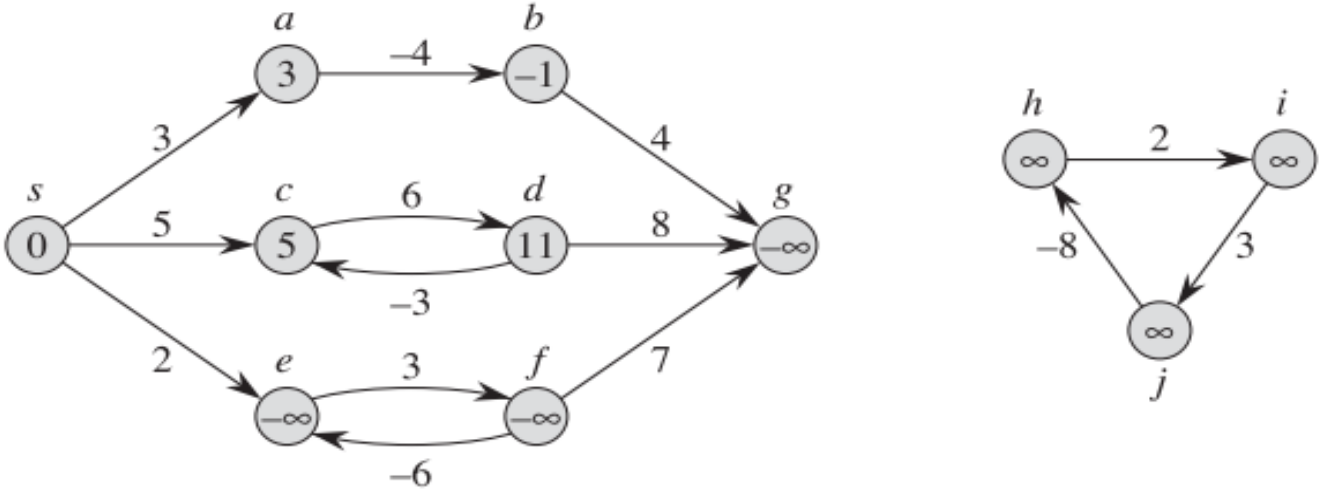
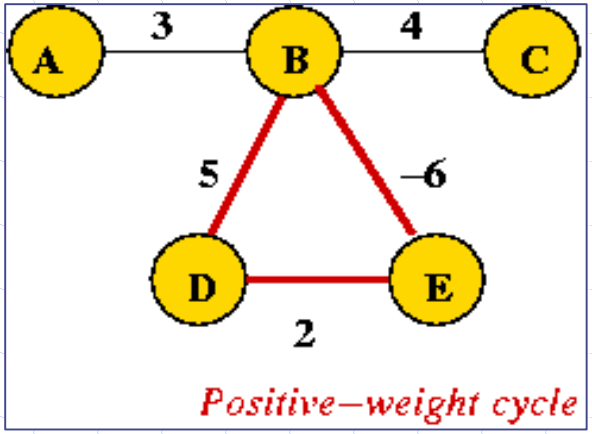



**Figure 24.1** Negative edge weights in a directed graph. The shortest-path weight from source $s$ appears within each vertex. Because vertices $e$ and $f$ form a negative-weight cycle reachable from $s$, they have shortest-path weights of $-\infty$. Because vertex $g$ is reachable from a vertex whose shortest-path weight is $-\infty$, it, too, has a shortest-path weight of $-\infty$. Vertices such as $h$, $i$, and $j$ are not reachable from $s$, and so their shortest-path weights are $\infty$, even though they lie on a negative-weight cycle.

# Shortest Path Problem (Cont.)

**Can a shortest path contain a cycle?**

- It can not contain a positive-weight cycle.
- When removing the cycle, it produces a new path with the same source and destination vertices and a lower path weight.




- Thus, we assume it has no cycles (simple path).
  - It contains at most |V| distinct vertices and |V|-1 edges.

# Algorithms for solving the shortest path problem

1. The Dijkstra's algorithm (Dijkstra published the algorithm in 1959, two years after Prim)
   - Assumes that *all* edge weights in the input graph are nonnegative.
2. The Bellman-Ford algorithm (The algorithm was first proposed by Alfonso Shimbel (1955), but is instead named after Richard Bellman and Lester Ford Jr., who published it in 1958 and 1956, respectively)
   - Allow negative-weight edges in the input graph and produce a correct answer as long as *no* negative-weight cycles are reachable from the source.

# Relaxation

- For each vertex v ∈ V, we maintain an attribute *v.d*, which is an upper bound on the weight of a shortest path from source *s* to *v*. We call *v.d* a shortest-path estimate.
- We initialize the shortest-path estimates and predecessors by the following $\theta$(V) time procedure:

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

# Relaxation (cont.)

- The process of relaxing an edge $(u,v)$ consists of testing whether we can improve the shortest path to $v$ found so far by going through $u$ and, if so, updating $v.d$ and v. $\pi$ .
- A relaxation step may decrease the value of the shortest-path estimate $v.d$ and update v's predecessor attribute $v.\pi$.

# Relaxation (cont.)

- The following code performs a relaxation step on edge (u,v) in O(1) time.

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2        v.d = u.d + w(u, v)
3        v.π = u
```

# Relaxation (cont.)

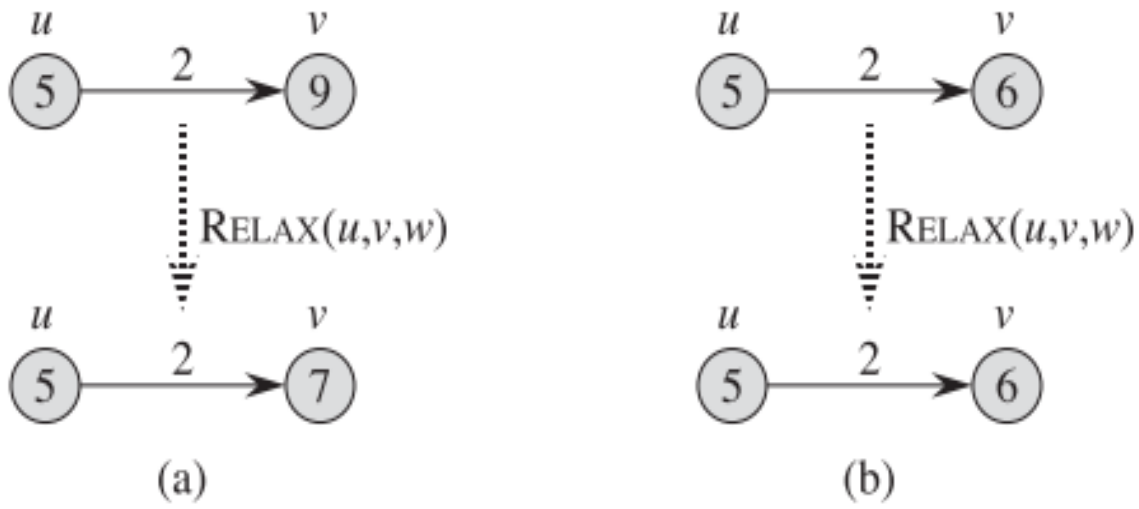



**Figure 24.3** Relaxing an edge $(u, v)$ with weight $w(u, v) = 2$. The shortest-path estimate of each vertex appears within the vertex. **(a)** Because $v.d > u.d + w(u, v)$ prior to relaxation, the value of $v.d$ decreases. **(b)** Here, $v.d \leq u.d + w(u, v)$ before relaxing the edge, and so the relaxation step leaves $v.d$ unchanged.

# Dijkstra Algorithm

- Dijkstra's algorithm maintains a set S of vertices whose final shortest-path weights from the source s have already been determined.

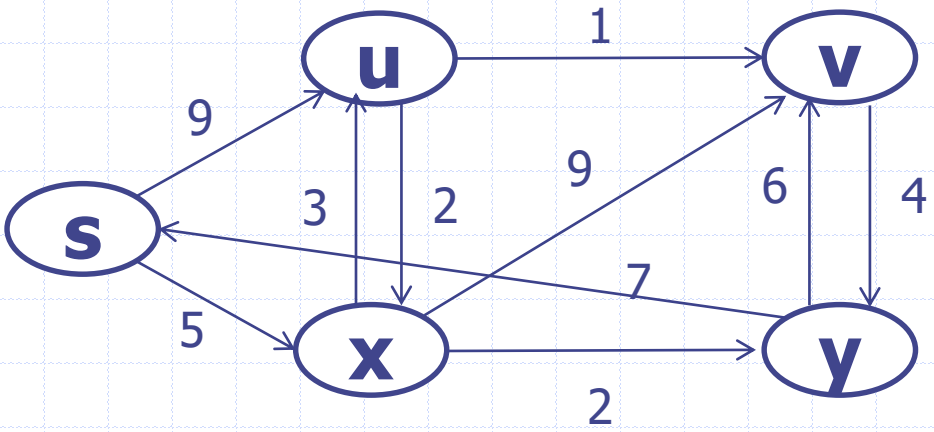
```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

# Dijkstra Algorithm: example

**Initially**



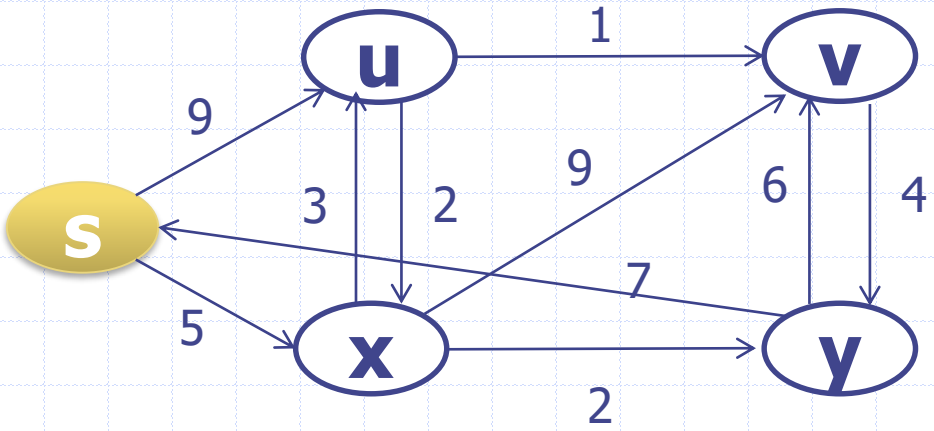

| v | s | u | x | v | y |
|---|---|---|---|---|---|
| **v.d** | 0 | ∞ | ∞ | ∞ | ∞ |
| **v.$\pi$** | nil | nil | nil | nil | nil |

Q = | s | u | v | x | y |

S = ∅

# Dijkstra Algorithm: example

**Step 1:**




| v | s | u | x | v | y |
|---|---|---|---|---|---|
| **v.d** | 0 | ∞ | ∞ | ∞ | ∞ |
| **v.π** | nil | nil | nil | nil | nil |

Extract-min = s

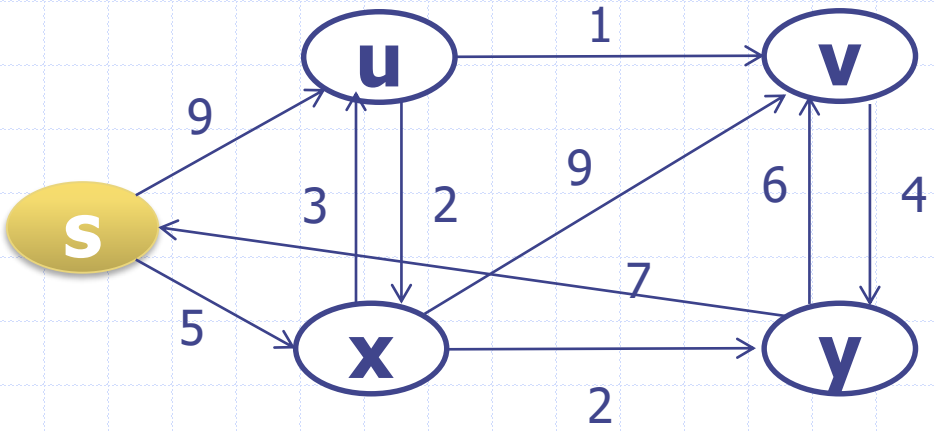
S= {s}

Q = | u | v | x | y |

adj{s} = { u, x}

# Dijkstra Algorithm: example

**Step 1:**




| v | s | u | x | v | y |
|---|---|---|---|---|---|
| **v.d** | 0 | 9 | 5 | ∞ | ∞ |
| **v.π** | nil | s | s | nil | nil |

Extract-min = s

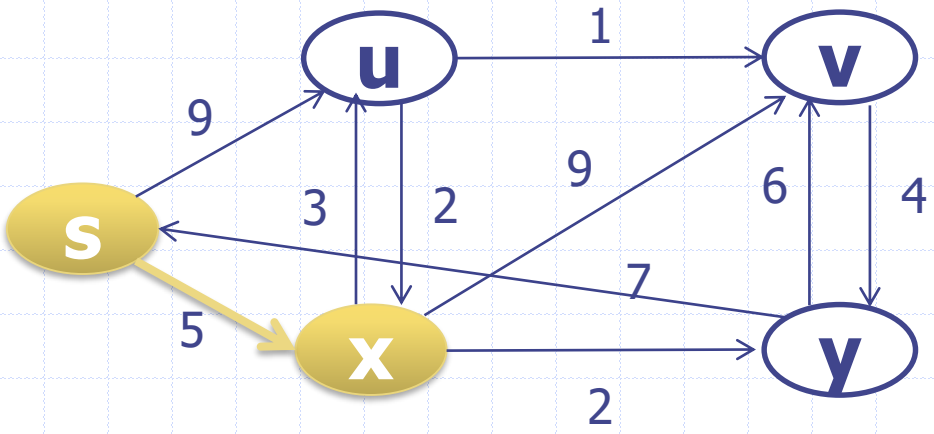
S= {s}

Q = | u | v | x | y |

adj{s} = { u, x}

# Dijkstra Algorithm: example

**Step 2:**




| V | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 9 | 5 | ∞ | ∞ |
| v.$\pi$ | nil | s | s | nil | nil |

Extract-min = x

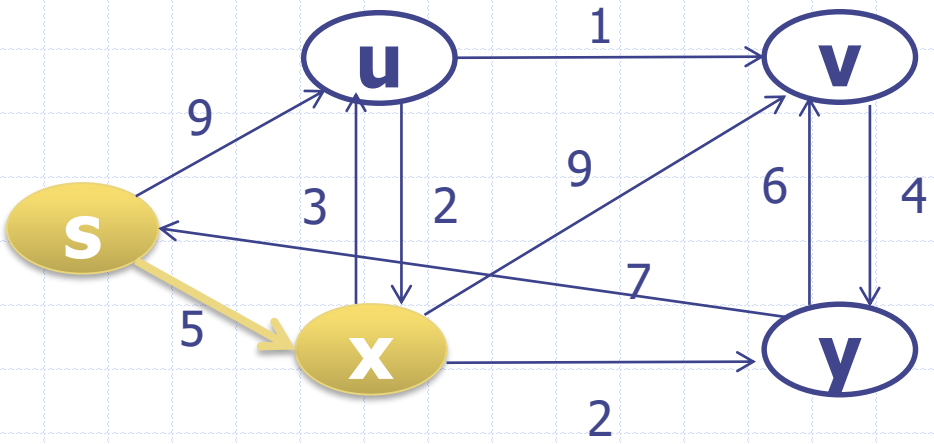
S = {s , x}

Q = | u | v | y |

adj{x} = {u , v, y}

# Dijkstra Algorithm: example

**Step 2:**




| v | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 8 | 5 | 14 | 7 |
| $v.\pi$ | nil | x | s | x | x |

Extract-min = x

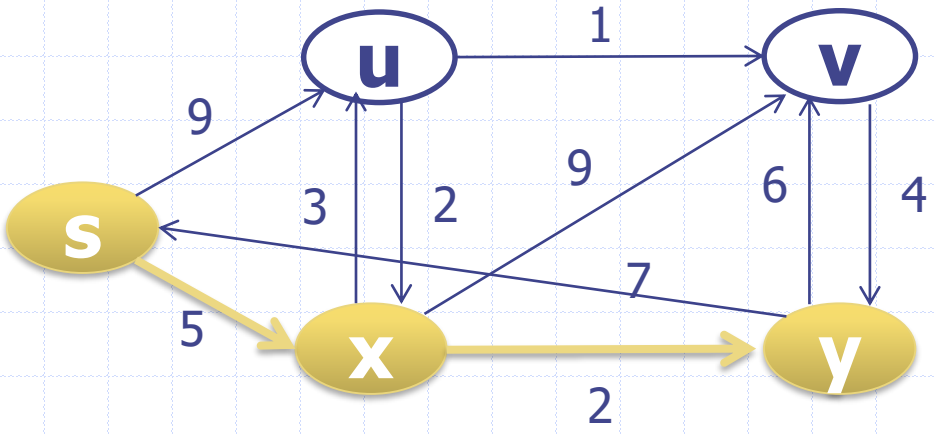
S = {s , x}

Q = | u | v | y |

adj{x} = {u , v, y}

# Dijkstra Algorithm: example

**Step 3:**




| v | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 8 | 5 | 14 | 7 |
| $v.\pi$ | nil | x | s | x | x |

Extract-min = y

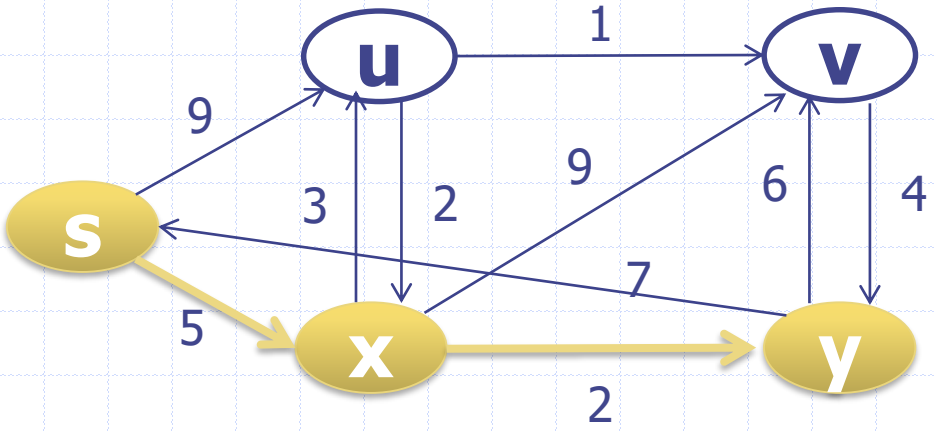
S = {s , x , y}

Q = | u | v |

adj{y} = {v , s}

# Dijkstra Algorithm: example

**Step 3:**




| V | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 8 | 5 | 13 | 7 |
| v.$\pi$ | nil | x | s | y | x |

Extract-min = y

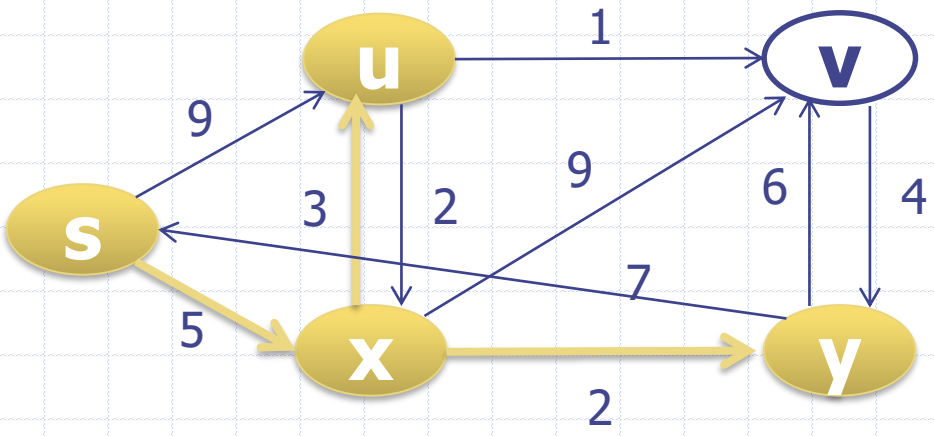
S = {s , x , y}

Q = | u | v |

adj{y} = {v , s}

# Dijkstra Algorithm: example

**Step 4:**




| V | s | u | x | v | y |
|---|---|---|---|---|---|
| **v.d** | 0 | 8 | 5 | 13 | 7 |
| **v.$\pi$** | nil | x | s | y | x |

Extract-min = u

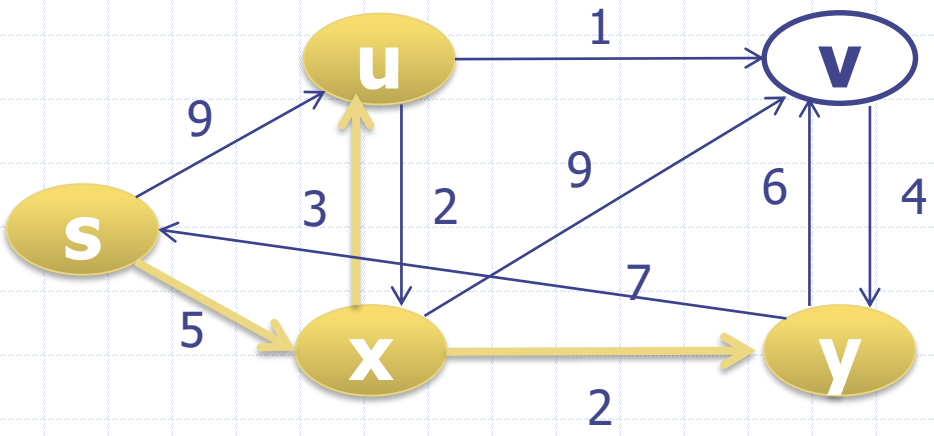
Q = | v |

S = {s , x , y, u}

adj{u} = {v , x}

# Dijkstra Algorithm: example

**Step 4:**




| V | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 8 | 5 | 9 | 7 |
| v.$\pi$ | nil | x | s | u | x |

Extract-min = u

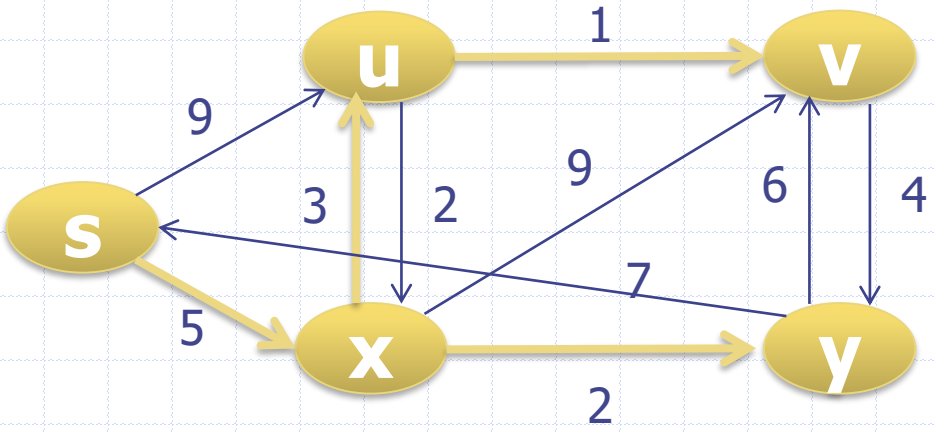
S = {s , x , y, u}

Q = | v |

adj{u} = {v , x}

# Dijkstra Algorithm: example

**Step 5:**




| v | s | u | x | v | y |
|---|---|---|---|---|---|
| v.d | 0 | 8 | 5 | 9 | 7 |
| v.$\pi$ | nil | x | s | u | x |

Extract-min = v

S = {s , x , y, u, v}

Q = ∅

adj{v} = {y}

# Analysis of Dijkstra's Algorithm

- The running time depends on how we implement the min-priority queue Q.
  - Array
  - Min-heap

# Analysis of Dijkstra's Algorithm

1. if we build Q as an array:

- Line 3: O(V), implicit insert operation per vertex takes O(1)
- Line 4: the while takes O(V).
- Line 5: Extract-Min() operation takes O(V).
- Line 6: added all vertexes exactly once to set S takes O(V)
- Line 7: for loop takes O(E) to examine each edge exactly once
- Line 8: implicit DECREASE-KEY operation takes O(1).

Total time is $O(V^2 + E) = O(V^2)$

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

# Analysis of Dijkstra's Algorithm

2. if we build Q as a min-heap:

- Line 3: implicit insert operation by building the min-heap takes O(V)
- Line 4: the while takes O(V).
- Line 5: Extract-Min() operation takes O(lg V)
- Line 6: added all vertexes exactly once to set S takes O(V)
- Line 7: for loop takes O(E) to examine each edge exactly once
- Line 8: implicit DECREASE-KEY operation takes O(lg V)

Total time is O((V+E) lg V)=O(E lg V).

```
RELAX(u, v, w)
1  if v.d > u.d + w(u, v)
2      v.d = u.d + w(u, v)
3      v.π = u
```

```
INITIALIZE-SINGLE-SOURCE(G, s)
1  for each vertex v ∈ G.V
2      v.d = ∞
3      v.π = NIL
4  s.d = 0
```

```
DIJKSTRA(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  S = ∅
3  Q = G.V
4  while Q ≠ ∅
5      u = EXTRACT-MIN(Q)
6      S = S ∪ {u}
7      for each vertex v ∈ G.Adj[u]
8          RELAX(u, v, w)
```

# Bellman-ford Algorithm

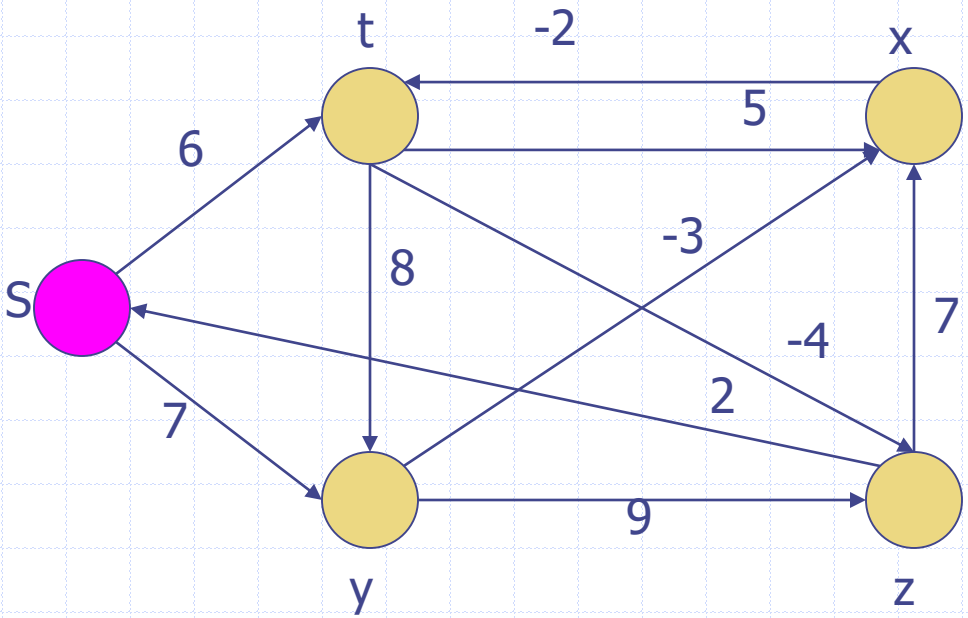
- Allows negative-weight edges.
- Computes d[v] and $\pi$[v] for all v $\in$ V .
- Returns TRUE if no negative-weight cycles reachable from s, FALSE otherwise.
- The algorithm makes |V|-1 passes.
  - Bellman Ford's Algorithm is slower than Dijkstra's algorithm
- Each pass relaxes all edges of the graph.

# Bellman-ford Algorithm

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```

# Bellman-Ford Algorithm - Example

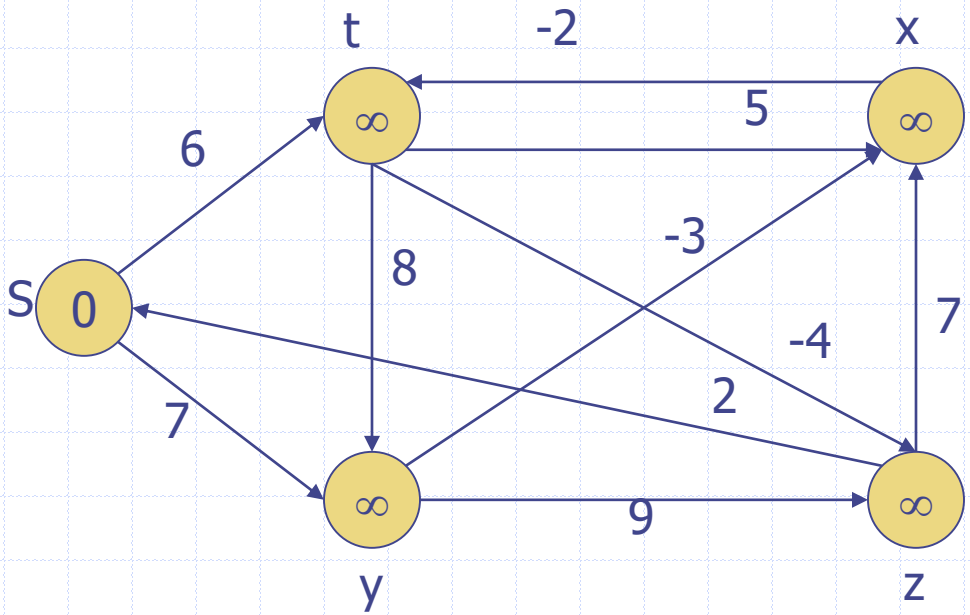
Find the shortest path from S to all vertices in the graph :



# Bellman-Ford Algorithm - Example

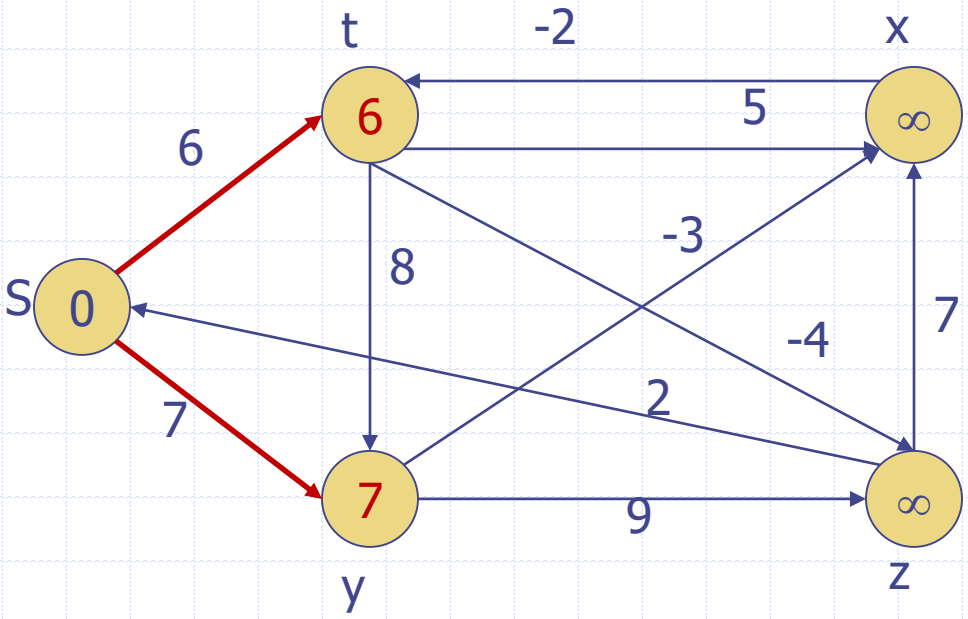
Initialize all vertices :

- In this particular example, each pass relaxes the edges in the order :
  (t , x), (t , y) , (t , z), (x , t), (y, x), (y , z), (z , x), (z , s), (s , t), (s , y)



# Bellman-Ford Algorithm - Example

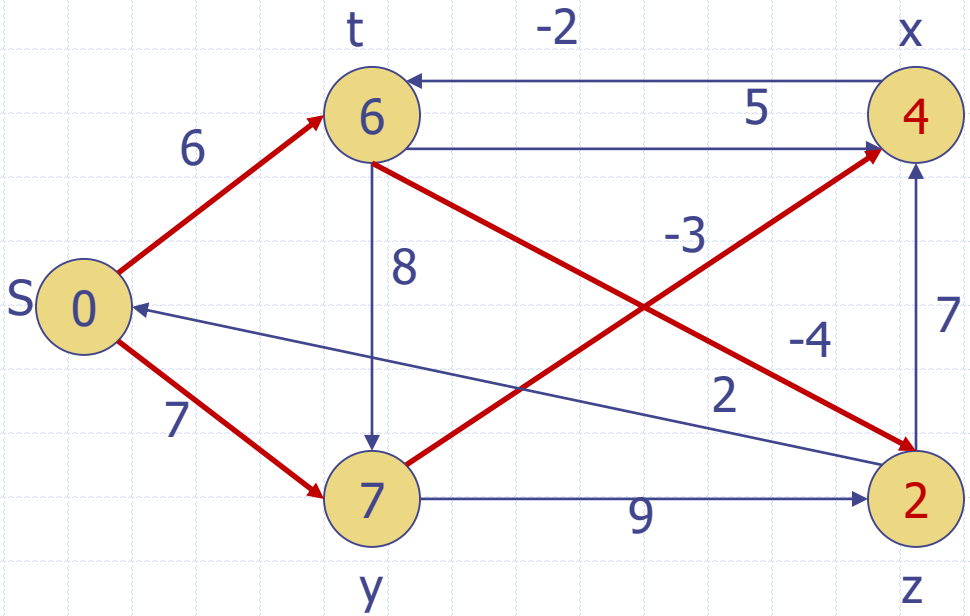
- In this particular example, each pass relaxes the edges in the order :
  (t , x), (t , y) , (t , z), (x , t), (y, x), (y , z), (z , x), (z , s), (s , t), (s , y)
- **First Pass :**



# Bellman-Ford Algorithm - Example

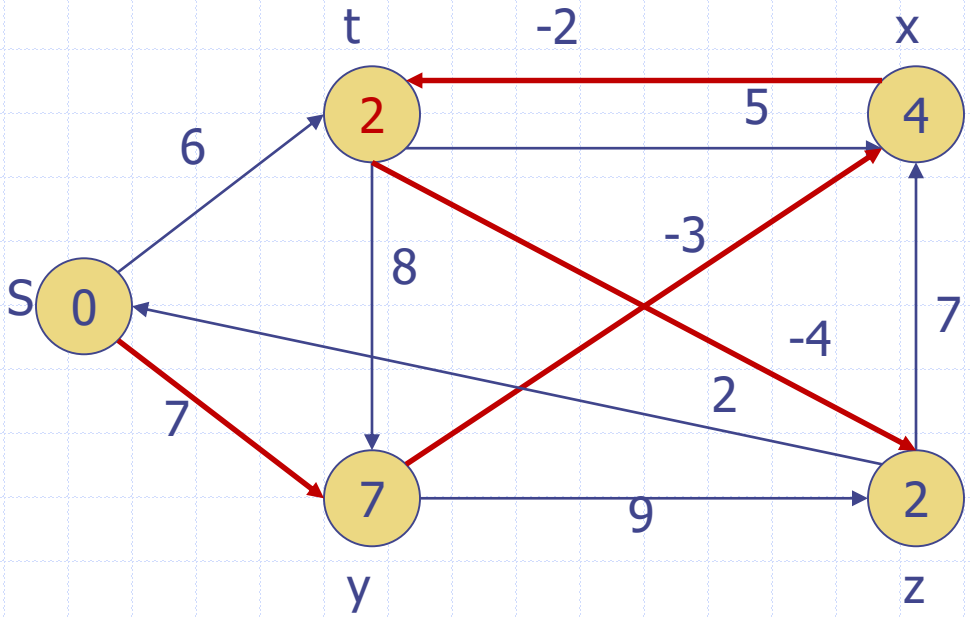
- In this particular example, each pass relaxes the edges in the order :
  (t , x), (t , y) , (t , z), (x , t), (y , x), (y , z), (z , x), (z , s), (s , t), (s , y)

- **Second Pass :**



# Bellman-Ford Algorithm - Example

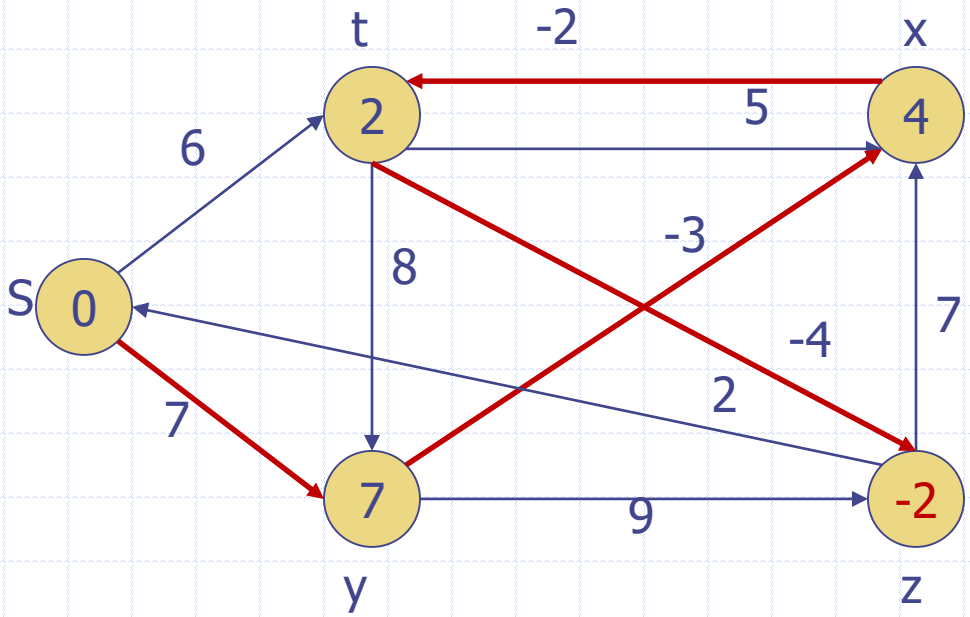
- In this particular example, each pass relaxes the edges in the order:
  (t , x), (t , y) , (t , z), (x , t), (y , x), (y , z), (z , x), (z , s), (s , t), (s , y),
- **Third Pass :**



# Bellman-Ford Algorithm - Example

- In this particular example, each pass relaxes the edges in the order :
  (t , x), (t , y) , (t , z), (x , t), (y , x), (y , z), (z , x), (z , s), (s , t), (s , y),

- **Fourth Pass :**



# Analysis of Bellman-ford Algorithm

The Bellman-Ford algorithm runs in time O(VE) since:

- the initialization in line 1 takes O(V) time
- each of the |V|-1 passes over the edges in lines 2–4 takes O(E) time
- and the for loop of lines 5–7 takes O(E) time.

```
BELLMAN-FORD(G, w, s)
1  INITIALIZE-SINGLE-SOURCE(G, s)
2  for i = 1 to |G.V| − 1
3       for each edge (u, v) ∈ G.E
4            RELAX(u, v, w)
5  for each edge (u, v) ∈ G.E
6       if v.d > u.d + w(u, v)
7            return FALSE
8  return TRUE
```